SIMATS ENGINEERING

Department of _____

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

Functions and Object-Oriented Programming Concepts

Particulars	Details
Student Name	___Pavithra Sri S_____
Register Number	___192572079_____
Department / Section	__CSE with AI___
Academic Year / Semester	___2026____
Faculty Name	__Amirtha_____
Nature of Assignment	Individual
Duration	1 Week
Total Marks	50
Submission Date	31/10/2026
GitHub Repository	DSA01-<RegisterNumber>-Assignment2
Compiler	g++ (C++17)

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts including function prototyping, references, inline functions, default arguments, overloading, classes, access specifiers, arrays in classes, static members, arrays of objects and friend functions.

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
Q1	Call by Value, Reference and Pointer	6		
Q2	Return by Reference and Its Dangers	4		
Q3	Default Arguments and Function Overloading	6		
Q4	Class with an Array Inside It	6		
Q5	Static Members and Arrays of Objects	7		
Q6	Friend Functions and Inline Functions	8		
Q7A	Diagnose the Limitation	2		
Q7B	Define the Design Objective	2		
Q7C	Design the Revised Solution	4		
Q7D	Predict Before Implementing	1		
Q7E	Implement and Validate	2		
Q7F	Defend the Final Design	2		
	TOTAL	50		

OVERALL REMARKS

Student Signature: ___Pavithra Sri S_____ Faculty Signature: _____

Program Director: _____

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

Assignment Title: Functions and Object-Oriented Programming Concepts

Nature: Individual Duration: 1 Week Total Marks: 50 Bloom's Level: K5 – Evaluate

Coverage

This assignment applies call by value, call by reference, pointers, return by reference, default arguments, function overloading, class design, arrays within classes, static members, arrays of objects, friend functions and inline functions.

Execution and GitHub Submission

- Create the repository DSA01-<RegisterNumber>-Assignment2 and share it with the faculty.
- Maintain separate folders Task01 to Task07 containing source code, input files and output screenshots.
- Add a README.md with student details, compiler version, exact compilation command and run instructions.
- Compile using g++ -std=c++17 -Wall -Wextra filename.cpp -o program.
- Commit work progressively, with at least one meaningful commit for each task.

Note: Outputs shown below are derived from the supplied code and the stated design assumptions. Timing results are not fabricated; run the benchmark on the submission computer and enter the measured values in the evidence section.

Q1 – CALL BY VALUE, BY REFERENCE AND BY POINTER (6 MARKS)

Ans: Hostel Fee Concession Updater

The three parameter-passing methods differ in whether the function receives an independent copy or access to the caller's object. The hostel fee is initially Rs 45,000 and the concession is 10%.

Method	Parameter form	Where it operates	Effect on caller
Call by value	long long fee	Local copy of the argument	Caller's fee remains unchanged.
Call by reference	long long& fee	Alias for the caller's variable	Changes the caller's fee.
Call by pointer	long long* fee	Pointer stores the caller variable's address	Changes caller's fee after dereferencing.

Three implementations

```
void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void applyByReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
void applyByPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= *fee * percent / 100;
}
```

Predicted output for the supplied program

```
1 12 13
1 100
16 13 6
```

Explanation: change(a,b,&c) increments local a only, while b and *c refer to the caller's variables. Therefore a remains 1, b becomes 12 and c becomes 13. bigger(a,c) returns a reference to the larger variable; because c is larger, the returned reference refers to c. Assigning 100 therefore changes c. show(1) uses default y=5 and z=10; show(1,2) uses default z=10; show(1,2,3) uses no defaults.

Complete the Code – TODO 1

```
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
```

Expected and actual output:

```
after call by value   : 45000
after call by reference: 40500
```

Call by value changes only the local parameter copy, so the original hostelFee remains 45,000. Call by reference aliases hostelFee and updates it to 40,500.

Const reference parameter

A const reference is suitable when a large object must be read without copying but the function must not modify it. Example: void printStudent(const Student& s). It avoids an expensive copy and prevents accidental changes through that parameter.

Design decision and alternative concept

For several concessions, returning the calculated fee is often easier to test and makes the update explicit. A reference-based function is appropriate when in-place modification is part of the interface. A value-returning version is:

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// Usage:
hostelFee = applyConcession(hostelFee, 10);
```

Parameter method	Time per call	Parameter-copy cost	Safety / use
Value	O(1)	Copies the long long value; a large Student object copies all its fields.	Safe isolation, but changes do not reach caller.
Reference	O(1)	No object copy; reference aliases original.	Efficient for intended modification.
Const reference	O(1)	No object copy.	Best for read-only large objects.

All three have O(1) time for this fixed arithmetic operation. A large object passed by value requires O(size of object) copying, while reference and const reference avoid that copy. A returned long long is also O(1).

Q2 – RETURN BY REFERENCE AND ITS DANGERS (4 MARKS)

Ans: Examination Marks Register

Returning a reference to an array element allows the caller to update the original element. The returned reference is an alias, not a new copy.

```
int& highestMark(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
const int& highestMarkReadOnly(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
int highestMarkCopy(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
```

Example usage

```
int marks[] = {78, 91, 84, 88};
highestMark(marks, 4) += 2;       // modifies marks[1] to 93
const int& top = highestMarkReadOnly(marks, 4);
int copiedTop = highestMarkCopy(marks, 4);
```

The expression highestMark(marks,4) can appear on the left side of an assignment because it returns an lvalue reference to an existing array element.

Dangling reference – unsafe function

```
int& invalidMark() {
    int local = 90;
    return local;  // WRONG: local is destroyed when function returns
}
```

A compiler commonly warns that a reference to a local variable is returned. After the function returns, local no longer exists. The reference is dangling, and reading or writing through it causes undefined behaviour. The exact diagnostic depends on the compiler and warning options.

Reference, const reference and copy

Return form	Caller can modify original?	Main property
int&	Yes	Direct alias to the array element.
const int&	No, not through this reference	Read-only alias; avoids copying.
int	No	Independent value copy.

Changed condition: std::vector

When a std::vector grows beyond its capacity, it may allocate a new memory block and move its elements. References, pointers and iterators to its old elements can then become invalid. A reference returned before a growth operation must not be used after a reallocation.

Alternative concept: return an index or pointer

```
int highestIndex(const int a[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (a[i] > a[best]) best = i;
    return best;
}
// Usage:
int i = highestIndex(marks, 4);
marks[i] += 2;

int* highestPointer(int a[], int n) {
    return &a[highestIndex(a, n)];
}
// Usage:
*highestPointer(marks, 4) += 2;
```

Finding the maximum takes O(n) time and O(1) extra space for reference, index and pointer versions. An index is generally easier to revalidate after vector growth, provided it remains within the new size. A pointer or reference may be invalidated by reallocation. A copied value is safest from invalidation but cannot update the stored mark.

Q3 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING (6 MARKS)

Ans: Courier Charge Calculator

The original declarations are ambiguous because charge(double weight) and the three-parameter overload with default values can both accept charge(2.0). The two-parameter overload can also accept a call with two arguments, while the third overload can supply the missing argument. The compiler cannot select a unique best function in such cases.

Redesigned overload set

```
struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

double charge(double weightKg,
              bool express = false,
              bool insured = false) {
    double amount = weightKg * 40.0;
    if (express) amount += 50.0;
    if (insured) amount += 20.0;
    return amount;
}

// Weight supplied in grams as an integer.
double charge(int weightGrams) {
    return charge(static_cast<double>(weightGrams) / 1000.0);
}

// Parcel-specific overload.
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}
```

The standalone one-parameter double overload is removed. The double overload now has optional express and insured arguments. The int overload means an integer is interpreted as grams, while a double is interpreted as kilograms. Parcel is passed by const reference to avoid copying and to prevent modification.

Six calls and predicted selection

Call	Selected overload	Reason
charge(2)	charge(int)	Exact match for int; 2 grams.
charge(2.0f)	charge(double,bool,bool)	float promotes to double; better than float-to-int conversion.
charge(2.0)	charge(double,bool,bool)	Exact double match; default Boolean arguments supplied.
charge(2500)	charge(int)	Exact int match; interpreted as 2,500 grams.
charge(2.5, true)	charge(double,bool,bool)	Exact double and bool matches;

		insured defaults false.
charge(Parcel{2.0,true,true})	charge(const Parcel&)	Exact Parcel argument match.

Assuming the illustrative rate of Rs 40 per kg, express fee Rs 50 and insurance fee Rs 20, the values are computed by the selected overload. The question primarily assesses overload selection; the example rates are assumptions for demonstrating the design.

How overload resolution works

- Exact match is preferred over a promotion or conversion.
- Promotion, such as float to double, is preferred over a general conversion such as floating-point to int.
- If no unique best viable function exists, the call is ambiguous and compilation fails.
- Default arguments fill omitted trailing parameters; they do not create a different overload.

Default arguments must be supplied from right to left. A declaration such as void f(int a=1, int b); is invalid because a later parameter without a default follows one with a default.

Alternative concept: one Parcel structure

```
struct ParcelOptions {
    double weightKg{};
    bool express{false};
    bool insured{false};
};
double parcelCharge(const ParcelOptions& p) {
    double total = p.weightKg * 40.0;
    if (p.express) total += 50.0;
    if (p.insured) total += 20.0;
    return total;
}
```

The structure-based design has one function signature and avoids overload ambiguity. Adding a new option usually means adding a field and updating the calculation, whereas overloads may require additional signatures. Overload resolution happens at compile time and adds no run-time dispatch cost. Both designs perform a fixed number of calculations in O(1) time. Passing ParcelOptions by const reference avoids copying; passing it by value copies its fields, which is still O(1) for this small structure.

Q4 – SPECIFYING A CLASS WITH AN ARRAY INSIDE IT (6 MARKS)

Ans: Savings Account Mini-Statement

Each SavingsAccount object stores its account number, current balance, a fixed array of the latest 10 transactions and a count. All data is private so outside code cannot directly change the balance or corrupt transaction history.

```
#include <iostream>
#include <string>
class SavingsAccount {
private:
    long long accountNo;
    long long balancePaise;
    long long transactions[10];
    int count;

    void addTransaction(long long amount);

public:
    SavingsAccount(long long no, long long openingBalance)
        : accountNo(no), balancePaise(openingBalance), count(0) {}

    void deposit(long long amount);
    bool withdraw(long long amount);
    void miniStatement() const;
};

void SavingsAccount::addTransaction(long long amount) {
    if (count < 10) {
        transactions[count++] = amount;
    } else {
        for (int i = 1; i < 10; ++i)
            transactions[i - 1] = transactions[i];
        transactions[9] = amount;
    }
}

void SavingsAccount::deposit(long long amount) {
    if (amount <= 0) return;
    balancePaise += amount;
    addTransaction(amount);
}

bool SavingsAccount::withdraw(long long amount) {
    if (amount <= 0 || amount > balancePaise) return false;
    balancePaise -= amount;
    addTransaction(-amount);
    return true;
}
```

```
void SavingsAccount::miniStatement() const {
   std::cout << "Account: " << accountNo
             << " Balance: " << balancePaise << " paise\n";
   for (int i = 0; i < count; ++i)
      std::cout << transactions[i] << " paise\n";
}
```

Class design and access specifiers

Member	Access	Reason
accountNo, balancePaise	private	Protect account identity and balance.
transactions[10], count	private	Prevent external corruption of history.
deposit(), withdraw(), miniStatement()	public	Provide controlled account operations.
addTransaction()	private	Internal helper used by public operations.

Some member functions are declared inside the class and defined outside using SavingsAccount::, demonstrating the scope resolution operator. Functions defined inside a class are implicitly inline candidates, but the compiler is not required to substitute their body at every call.

Handling the 11th transaction

With shifting, when the array is full, elements 1 through 9 are copied one position left and the new transaction is placed at index 9. This discards the oldest transaction and costs O(k) time, where k is the number of stored entries (10 here).

A circular buffer keeps a start index. When full, it writes the new value over the oldest slot and advances start modulo capacity. Insertion takes O(1) time. Printing the statement must traverse in chronological order using (start+i)%capacity.

Circular-buffer alternative

```
// Core insertion logic for capacity 10:
transactions[next] = amount;
next = (next + 1) % 10;
if (count < 10) ++count;
// Oldest item is at (next - count + 10) % 10.
```

A std::deque<long long> is another option: push_back(newValue), then pop_front() if size exceeds 10. Both circular buffer and deque insertion/removal at the ends are O(1) in the intended operations. A fixed array uses O(10) space and no dynamic allocation; deque uses O(k) storage with container overhead.

Design decision for 50 transactions

Choose a circular buffer with a named capacity constant set to 50. The class interface remains unchanged; only the capacity and internal insertion/iteration logic change. A deque is easier to maintain if capacity changes frequently, while a fixed circular buffer offers predictable storage and no per-entry dynamic allocation. Both use O(k) space.

Q5 – STATIC MEMBERS AND ARRAYS OF OBJECTS (7 MARKS)

Ans: College Bus Seat Tickets

The seat counter and bus capacity belong to the class as a whole, not to one ticket. Therefore issued and CAPACITY are static members. Each ticket stores its own seat number and passenger name.

Static member concepts

- A static data member has one shared copy for the entire class, regardless of the number of objects.
- A static data member is declared inside the class and, in the traditional out-of-class form used here, defined once outside the class.
- A static member function has no this pointer because it is not called for one particular object.
- A static member function can directly access static members, but cannot directly access a non-static member because no particular object is identified.
- An array of objects creates each element and calls its constructor. A loop-created local array is destroyed when the function ends, but the class static counter retains its value.

Predict the output – Token program

```
101 103 104 4
```

Token::next begins at 100. Constructing counter[3] calls the constructor three times and assigns IDs 101, 102 and 103. Constructing extra calls it once more and assigns 104. Thus issued() returns next-100 = 4.

Complete the Code – BusTicket

```
class BusTicket {
   static int issued;
   static const int CAPACITY = 3;
   int seat;
   std::string name;
public:
   explicit BusTicket(const std::string& n) : seat(0), name(n) {
      if (issued < CAPACITY) {
         ++issued;
         seat = issued;
      }
   }
   static int count() {
      return issued;
   }
   void print() const {
      std::cout << name << ": "
            << (seat ? "Seat " + std::to_string(seat)
                  : std::string("Waiting")) << '\n';
   }
};
int BusTicket::issued = 0;
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
```

```
Issued: 3
```

Creating tickets inside a function called twice

```
void createTrip(const std::string& passenger) {
    BusTicket ticket(passenger);
    ticket.print();
}
// Calling createTrip twice creates two ticket objects,
// but the static issued counter is shared and is not reset.
```

If the function is called twice, the second call sees the updated static count. For example, if the first call issues one seat, the next ticket receives seat 2. Once issued reaches 3, subsequent tickets are placed on the waiting list. Destroying a ticket object does not automatically decrement the static counter.

Array of objects

```
BusTicket trip[4] = {
    BusTicket("Asha"), BusTicket("Bala"),
    BusTicket("Chitra"), BusTicket("Dev")
};
for (const BusTicket& ticket : trip)
    ticket.print();
```

Alternative concept: global counter or TicketOffice

A global counter can provide O(1) access but weakens encapsulation and can accidentally be shared by unrelated buses. A TicketOffice object can own a counter and be passed to each ticket; this makes separate buses easy to model. Static members also have O(1) time and one shared counter, but a single static counter per class cannot independently track two buses unless the class is redesigned to maintain separate office instances.

Approach	Time	Space	Assessment
Static member	O(1) issue operation	One counter per class	Simple for one bus; shared state.
Global variable	O(1) issue operation	One global counter	Least encapsulated; global naming/state risk.
TicketOffice object	O(1) issue operation	One counter per office	Best for multiple buses and independent capacity.

Recommendation: use a TicketOffice object if the system may operate multiple buses. Keep seat issuance and capacity together in that object so each bus has an independent counter.

Q6 – FRIEND FUNCTIONS AND INLINE FUNCTIONS (8 MARKS)

Ans: Train Journey Planner

The Time class stores hours and minutes privately. The functions minutesBetween(a,b) and earlier(a,b) compare two Time objects equally. A non-member friend function is suitable when the operation is symmetric and should not be associated with only one operand.

Completed friend functions – TODO 1 and TODO 2

```
int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    int difference = first - second;
    return difference < 0 ? -difference : difference;
}

bool earlier(const Time& a, const Time& b) {
    if (a.h != b.h) return a.h < b.h;
    return a.m < b.m;
}
```

Expected output:

```
205 false
```

9:45 AM is 585 minutes after midnight. 1:10 PM is 790 minutes after midnight. Their absolute difference is 790−585 = 205 minutes. earlier(arrival, departure) asks whether 13:10 is earlier than 09:45, so the answer is false.

Correctness checks

Test	Expected result	Reason
Time(9,45), Time(13,10)	205; earlier(first,second)=true	Times are on either side of noon.
Time(13,10), Time(9,45)	205; earlier(first,second)=false	Reverse ordering.
Time(12,00), Time(12,00)	0; earlier=false	Same time has zero difference.
Time(11,59), Time(12,01)	2; earlier(first,second)=true	Two-minute interval across noon.

Friend function versus member function

Feature	Friend function	Member function
Call syntax	minutesBetween(a,b)	a.minutesTo(b)
Access	Can access private members of the class that grants friendship.	Can access private members of its own object.
Symmetry	Treats both operands equally.	Has an implicit left-hand object.
Encapsulation	Grants specific external function access.	Keeps operation inside the class interface.

A friend function is not a member of the class even though it is declared inside the class with the friend keyword. It is called like an ordinary non-member function. Making h and m public would remove the need for friendship but expose internal representation to all callers, weakening encapsulation.

Inline functions and performance

The inline keyword requests that the compiler consider expanding a function body at the call site; it does not force the compiler to do so. Functions defined inside a class are implicitly inline candidates. Small functions such as a simple getter may benefit, but modern compilers make their own optimisation decisions.

The assignment asks for a benchmark of 10,000,000 calls at -O0 and -O2. Do not claim timings without executing the benchmark. At -O0, function-call overhead may be more visible. At -O2, the compiler may inline or optimise away equivalent calls, so the difference may become very small. Ensure the benchmark result is used (for example, accumulate into a printed checksum) to prevent the compiler from removing the work.

Alternative concept: member functions and getters

```
class Time {
private:
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    int totalMinutes() const { return h * 60 + m; }
    int minutesTo(const Time& other) const {
        int d = totalMinutes() - other.totalMinutes();
        return d < 0 ? -d : d;
    }
    bool isEarlier(const Time& other) const {
        return totalMinutes() < other.totalMinutes();
    }
};
```

A getter-based non-member implementation can also compare totalMinutes() values without friendship. All approaches perform O(1) work per comparison and use O(1) extra space. After inlining, their run-time performance is generally equivalent. Member functions provide a clear object-oriented interface; getters provide the narrowest access; friend functions are useful when a symmetric operation genuinely needs private access.

Deployment decision

For this small Time abstraction, prefer public const member functions or getters for ordinary comparisons. Keep friend functions only when symmetric non-member operations need access to private state. Do not make private fields public merely to avoid writing a friend declaration.

Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected program: Q4 – Savings Account Mini-Statement
The selected limitation is the shifting-array implementation used when the transaction history is full. Each new transaction after the first 10 requires moving the remaining nine elements. This is correct for a small history, but it repeats unnecessary data movement as the history capacity grows.

A. Diagnose the limitation (2 Marks)

When the fixed array is full, insertion shifts k−1 existing transactions one place to the left before storing the new transaction. Its time complexity is O(k). With a capacity of 10, at most nine moves are needed per new transaction. If the bank increases the history to 50, up to 49 moves are required for every new transaction after the buffer fills.

Capacity k	Maximum existing entries shifted per new transaction	Operation
10	9	O(k)
50	49	O(k)
100	99	O(k)

The limitation comes from preserving chronological order by physically moving all stored elements after every insertion.

B. Define the design objective (2 Marks)

Objective: redesign transaction insertion so that adding a transaction to a full history takes constant time O(1), while retaining only the most recent k transactions in chronological order. The solution must use fixed bounded storage and must not change the public deposit(), withdraw() and miniStatement() interface.

C. Design the revised solution (4 Marks)

Use a circular buffer with three fields: an array of k transactions, a start index identifying the oldest transaction, and a count of stored entries. When space is available, append at (start+count)%k. When full, overwrite the oldest entry and advance start by one modulo k.

```
class TransactionHistory {
private:
   static constexpr int CAPACITY = 50;
   long long data[CAPACITY]{};
   int start = 0;   // index of oldest stored transaction
   int count = 0;

public:
   void add(long long amount) {
      if (count < CAPACITY) {
         int pos = (start + count) % CAPACITY;
         data[pos] = amount;
         ++count;
      } else {
         data[start] = amount;            // overwrite oldest
         start = (start + 1) % CAPACITY;   // next oldest
```

```
        }
    }

    int size() const { return count; }

    void print() const {
        for (int i = 0; i < count; ++i) {
            int pos = (start + i) % CAPACITY;
            std::cout << data[pos] << '\n';
        }
    }
};
```

The SavingsAccount class can contain one TransactionHistory object and call history.add(amount) after a successful deposit or withdrawal. The circular buffer uses O(k) memory, the same asymptotic storage as the fixed array, and O(1) time per insertion. Printing all retained transactions takes O(k) time.

Circular buffer flow

```
Add transaction
     |
     v
Is count < CAPACITY?
  /        \
 Yes         No
  |          |
Write at    Overwrite data[start]
(start+count)  |
  |        start = (start+1) % CAPACITY
count++        |
  \         /
     Done
```

Trade-off: the internal array is no longer stored in chronological order from index zero. Printing must calculate the correct circular index. The extra start index is constant memory.

D. Predict before implementing (1 Mark)

Prediction: once the history is full, circular-buffer insertion will perform a fixed number of assignments and index updates, independent of capacity. Therefore insertion work will remain O(1), compared with O(k) shifts in the original design. The number of retained transactions and chronological output should remain identical.

E. Implement and validate (2 Marks)

Use the same input sequence for both implementations: insert transaction values 10, 20, 30, …, 600 into a history of capacity 50. After all 60 insertions, both versions must retain the most recent 50 values, from 110 through 600, in chronological order.

Validation item	Original shifting array	Circular buffer
Capacity	50	50
Input sequence	10 to 600 in steps of 10	Same sequence
Expected retained count	50	50

Expected oldest retained	110	110
Expected newest retained	600	600
Insertion complexity after full	O(k)	O(1)

For measurement, run many insertions (for example, 10 million) after initialising each structure, repeat the test several times and record the median duration. Use the same compiler, optimisation level and hardware. Keep a checksum or print the final oldest/newest values so the work cannot be optimised away. Enter the actual timings obtained:

Build / test	**Shifting-array measured time**	**Circular-buffer measured time**
g++ -std=c++17 -O0	_____ seconds	_____ seconds
g++ -std=c++17 -O2	_____ seconds	_____ seconds

F. Defend the final design (2 Marks)

The circular buffer meets the objective because a full-buffer insertion overwrites one element and advances one index without shifting the other entries. It preserves the required last-k history and uses the same O(k) storage order as the original fixed array. The original shifting design may still be preferable for a very small capacity when simplicity and direct chronological indexing are more important than insertion efficiency.

Final decision: use the circular buffer for a 50-transaction mini-statement because the history capacity is larger and each new transaction should be recorded with predictable constant-time work. The improvement should be confirmed using the same measured workload; exact timing values must come from the student's own execution.

FINAL SUMMARY

This assignment demonstrates how C++ functions and class features can be selected to solve practical problems. References and pointers allow controlled modification of caller data, while const references avoid unnecessary copies. Return-by-reference is efficient but must never refer to a destroyed local variable. Default arguments simplify common calls, and carefully designed overloads prevent ambiguity. Classes protect data through access specifiers, arrays and circular buffers maintain bounded histories, static members store class-wide state, and friend functions support symmetric operations that need private access. The final redesign uses a circular buffer to reduce transaction insertion from O(k) to O(1).

Submission Checklist

- Complete and execute all Q1–Q7 programs using g++ with the required warning flags.
- Write output predictions before executing the supplied Predict the Output programs.
- Capture actual output screenshots and compiler warnings where requested.
- Run the inline benchmark at -O0 and -O2 and record measured timings.
- Run the original and redesigned transaction-history implementations using identical inputs.
- Create Task01 to Task07 folders and include source code, test inputs, outputs and a README.md.
- Commit each task progressively and add the working GitHub repository link to the cover page.

Student note: The sample outputs are analytically derived. Replace blank timing fields with actual measurements from your computer and verify the compiler diagnostics before submission.